Video Streaming Service

Αναλυτική επεξήγηση κάθε αρχείου του repository
και θεωρία YAML, Docker και Kubernetes

Repository: RubenKingCP/Video-Streaming-Service
Οκτώβριος 2026

Περιεχόμενα

1. Επισκόπηση του έργου **4**

1.1 Αρχιτεκτονική 4

1.2 Δομή φακέλων 4

1.3 Τα endpoints της υπηρεσίας 4

2. Θεωρία: Πώς λειτουργεί το HLS streaming **6**

3. Θεωρία: Αρχεία YAML **7**

3.1 Τα τρία δομικά στοιχεία 7

3.2 Κανόνες εσοχής – η πιο συνηθισμένη πηγή λαθών 7

3.3 Τύποι και παγίδες 8

3.4 Πολλαπλά έγγραφα, πολυγραμμικό κείμενο, anchors 8

3.5 Πώς ελέγχεις ένα YAML 8

4. Θεωρία: Docker **9**

4.1 Βασικές έννοιες 9

4.2 Οι εντολές του Dockerfile 9

4.3 Build cache – γιατί έχει σημασία η σειρά 9

4.4 Πώς φτιάχνεις ένα Dockerfile από το μηδέν (συνταγή) 10

5. Θεωρία: Kubernetes **11**

5.1 Δηλωτικό μοντέλο (declarative) και reconciliation loop 11

5.2 Τα βασικά αντικείμενα 11

5.3 Η ανατομία κάθε manifest Kubernetes 11

5.4 Πώς φτιάχνεις τα YAML του Kubernetes από το μηδέν (συνταγή) 12

5.5 Πώς ταιριάζουν όλα μαζί: labels 13

6. Επεξήγηση κάθε αρχείου **14**

6.1 scripts/prepare_media.py – μετατροπή βίντεο σε HLS 14

6.2 backend/requirements.txt – εξαρτήσεις Python 15

6.3 backend/app/core/config.py – ρυθμίσεις 15

6.4 backend/app/services/video_service.py – λογική εύρεσης βίντεο 16

6.5 backend/app/api/routes/videos.py – endpoints βίντεο 18

6.6 backend/app/api/routes/health.py – έλεγχοι υγείας 19

6.7 backend/app/core/metrics.py – μετρικές Prometheus 20

6.8 backend/app/main.py – το σημείο εισόδου της εφαρμογής 22

6.9 backend/app/static/index.html – σελίδα demo 24

6.10 backend/app/static/app.js – λογική της σελίδας 24

6.11 backend/app/static/style.css – εμφάνιση 25

6.12 Dockerfile – κατασκευή του image 25

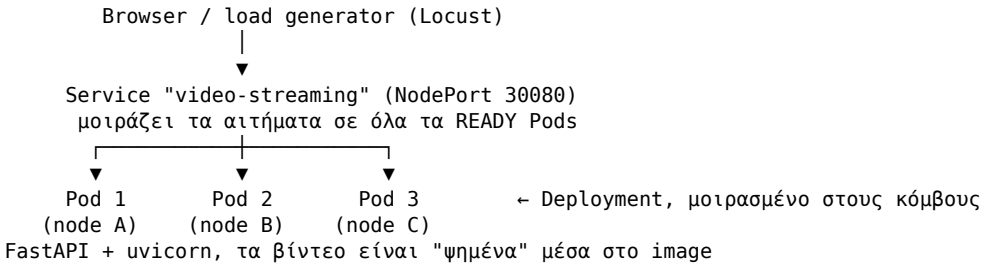
6.13 .dockerignore – τι ΔΕΝ μπαίνει στο build 26
6.14 k8s/deployment.yaml – το Deployment 26
6.15 k8s/service.yaml – το Service 28
6.16 scripts/locustfile.py – γεννήτρια φόρτου 29
6.17 media/ – τα βίντεο 30
6.18 .gitignore, README.md, LICENSE 30
7. Όλη η ροή από την αρχή μέχρι το τέλος **32**
7.1 Χρήσιμες εντολές 32
7.2 Συχνά προβλήματα 32

1. Επισκόπηση του έργου

Το έργο είναι μια **υπηρεσία streaming βίντεο** γραμμένη σε Python με το framework **FastAPI**. Τρέχει μέσα σε **container Docker** και αναπτύσσεται (deploy) σε ένα **cluster Kubernetes** με πολλούς κόμβους (nodes). Ο στόχος είναι να δειχθεί πώς μια απλή, *stateless* (χωρίς κατάσταση) υπηρεσία μπορεί να τρέχει σε πολλά πανομοιότυπα αντίγραφα (replicas), με τον Kubernetes να μοιράζει την κίνηση ανάμεσά τους, να ελέγχει την υγεία τους και να τα επανεκκινεί όταν χαλάσουν.

Τα βίντεο **δεν μετατρέπονται τη στιγμή του αιτήματος**. Έχουν ήδη μετατραπεί από πριν (με το script `prepare_media.py` και το εργαλείο ffmpeg) σε μορφή **HLS**: ένα αρχείο λίστας αναπαραγωγής (`index.m3u8`) και πολλά μικρά κομμάτια βίντεο (`seg_000.ts`, `seg_001.ts`, ...). Ο server απλώς στέλνει αρχεία μέσω HTTP. Γι' αυτό είναι γρήγορος, απλός και κλιμακώνεται εύκολα.

1.1 Αρχιτεκτονική



1.2 Δομή φακέλων

```
Video-Streaming-Service/
├── backend/
│   ├── requirements.txt            # Python βιβλιοθήκες
│   └── app/
│       ├── main.py                 # δημιουργεί την εφαρμογή FastAPI
│       ├── core/config.py          # ρυθμίσεις από μεταβλητές περιβάλλοντος
│       ├── core/metrics.py         # μετρικές Prometheus + middleware
│       ├── api/routes/health.py    # /health/live, /health/ready
│       ├── api/routes/videos.py    # /videos, /videos/{id}/{file}
│       ├── services/video_service.py# εύρεση αρχείων, προστασία path traversal
│       └── static/                 # σελίδα demo: index.html, style.css, app.js
├── k8s/deployment.yaml             # Kubernetes Deployment
├── k8s/service.yaml                # Kubernetes Service
├── media/source/                   # αρχικά βίντεο (π.χ. earth_spinning.mp4)
├── media/hls/<id>/                 # index.m3u8 + seg_NNN.ts
├── scripts/prepare_media.py        # μετατροπή βίντεο σε HLS με ffmpeg
├── scripts/locustfile.py           # γεννήτρια φόρτου (εικονικοί θεατές)
├── Dockerfile, .dockerignore, .gitignore, README.md, LICENSE
```

1.3 Τα endpoints της υπηρεσίας

Endpoint	Τι επιστρέφει	Αρχείο
`GET /`	Τη σελίδα demo με player HLS	main.py
`GET /videos`	JSON λίστα με τα διαθέσιμα βίντεο	videos.py
`GET /videos/{id}/index.m3u8`	Την playlist HLS ενός βίντεο	videos.py
`GET /videos/{id}/seg_NNN.ts`	Ένα κομμάτι (segment) του βίντεο	videos.py
`GET /health/live`	Liveness: η διεργασία ζει (200)	health.py

Endpoint	Τι επιστρέφει	Αρχείο
`GET /health/ready`	Readiness: υπάρχει βίντεο (200) ή όχι (503)	health.py
`GET /metrics`	Μετρικές σε μορφή Prometheus	main.py / metrics.py
`GET /docs`	Αυτόματη τεκμηρίωση API (Swagger) του FastAPI	—

2. Θεωρία: Πώς λειτουργεί το HLS streaming

Το **HLS (HTTP Live Streaming)** είναι πρωτόκολλο της Apple για μετάδοση βίντεο πάνω από απλό HTTP. Αντί να στέλνεται ένα τεράστιο αρχείο MP4, το βίντεο κόβεται σε μικρά κομμάτια λίγων δευτερολέπτων. Ο player κατεβάζει πρώτα μια **playlist** (αρχείο κειμένου .m3u8) που λέει ποια κομμάτια υπάρχουν και με ποια σειρά, και μετά κατεβάζει τα κομμάτια ένα-ένα, λίγο πριν τα χρειαστεί.

- **Πλεονεκτήματα:** δουλεύει με οποιονδήποτε web server, περνάει από firewalls/proxies (είναι απλό HTTP), η αναπαραγωγή ξεκινά αμέσως, και κάθε κομμάτι μπορεί να σερβιριστεί από *διαφορετικό* Pod — ιδανικό για load balancing.
- **.m3u8**: η playlist (κείμενο). **.ts**: MPEG Transport Stream, το δοχείο που περιέχει βίντεο H.264 και ήχο AAC.
- **VOD (Video On Demand)**: η playlist είναι πλήρης και τελειώνει με #EXT-X-ENDLIST (σε αντίθεση με το live).

Η playlist media/hls/sample/index.m3u8 γραμμή-γραμμή

```
#EXTM3U
#EXT-X-VERSION:3
#EXT-X-TARGETDURATION:8
#EXT-X-MEDIA-SEQUENCE:0
#EXT-X-PLAYLIST-TYPE:VOD
#EXTINF:8.333333,
seg_000.ts
#EXTINF:8.333333,
seg_001.ts
#EXTINF:8.333333,
seg_002.ts
#EXTINF:5.033333,
seg_003.ts
#EXT-X-ENDLIST
```

Γραμμή	Σημασία
`#EXTM3U`	Υποχρεωτική πρώτη γραμμή: δηλώνει αρχείο M3U playlist
`#EXT-X-VERSION:3`	Έκδοση του πρωτοκόλλου HLS
`#EXT-X-TARGETDURATION:8`	Μέγιστη διάρκεια κάθε segment σε δευτερόλεπτα
`#EXT-X-MEDIA-SEQUENCE:0`	Αριθμός του πρώτου segment
`#EXT-X-PLAYLIST-TYPE:VOD`	Βίντεο κατ' απαίτηση, η λίστα δεν θα αλλάξει
`#EXTINF:8.333333,`	Η διάρκεια του segment που ακολουθεί στην επόμενη γραμμή
`seg_000.ts`	Όνομα αρχείου segment (σχετικό URL ως προς την playlist)
`#EXT-X-ENDLIST`	Τέλος λίστας: δεν έρχονται άλλα segments

Παρατήρηση: το sample έχει segments ~8,3 δευτ. ενώ το προεπιλεγμένο του script είναι 4 δευτ. Ο ffmpeg κόβει μόνο σε keyframes· αν το βίντεο δημιουργήθηκε με άλλη ρύθμιση (π.χ. --segment 8) ή χωρίς force_key_frames, τα κομμάτια βγαίνουν μεγαλύτερα. Ο φάκελος test/ έχει 15 segments των 4 δευτ. (60 δευτ. test pattern).

3. Θεωρία: Αρχεία YAML

Το **YAML** (YAML Ain't Markup Language) είναι μια μορφή σειριοποίησης δεδομένων φτιαγμένη για να διαβάζεται εύκολα από ανθρώπους. Περιγράφει την ίδια δομή δεδομένων με το JSON (αντικείμενα, λίστες, αριθμούς, κείμενο), αλλά αντί για αγκύλες και εισαγωγικά χρησιμοποιεί **εσοχές (indentation)**. Στην πραγματικότητα κάθε έγκυρο JSON είναι και έγκυρο YAML. Το Kubernetes, το Docker Compose, το GitHub Actions και πολλά άλλα εργαλεία χρησιμοποιούν YAML για αρχεία ρυθμίσεων.

3.1 Τα τρία δομικά στοιχεία

Όλο το YAML χτίζεται από τρία πράγματα:

- **Scalar (απλή τιμή):** κείμενο, αριθμός, boolean, null. Π.χ. `replicas: 3`, `name: video-streaming`.
- **Mapping (λεξικό / αντικείμενο):** ζεύγη `κλειδί: τιμή`. Το κενό μετά την άνω-κάτω τελεία είναι **υποχρεωτικό**.
- **Sequence (λίστα):** στοιχεία που ξεκινούν με `-` (παύλα και κενό).

```
# Αυτό είναι σχόλιο (ό,τι ακολουθεί το # αγνοείται)
όνομα: Video Streaming        # scalar: κείμενο (χωρίς εισαγωγικά)
εκδοση: "1.0"                 # κείμενο με εισαγωγικά (αλλιώς θα γινόταν αριθμός 1.0)
replicas: 3                   # αριθμός (int)
ενεργό: true                  # boolean
τίποτα: null                  # κενή τιμή (ή ~)

ρυθμίσεις:                    # mapping μέσα σε mapping (εμφώλευση με εσοχή)
  cpu: 100m
  memory: 128Mi

θύρες:                        # sequence (λίστα) από απλές τιμές
  - 80
  - 8000

containers:                   # λίστα από mappings - το πιο συνηθισμένο μοτίβο στο Kubernetes
  - name: web
    image: nginx:1.27
  - name: sidecar
    image: busybox

flow_στυλ: {a: 1, b: [x, y]}  # "inline" γραφή, ίδια με JSON
```

3.2 Κανόνες εσοχής – η πιο συνηθισμένη πηγή λαθών

- Χρησιμοποιούνται **μόνο κενά (spaces), ποτέ Tab**. Ένα Tab κάνει το αρχείο άκυρο.
- Η σύμβαση είναι **2 κενά** ανά επίπεδο. Σημασία έχει τα στοιχεία του ίδιου επιπέδου να είναι **ακριβώς στην ίδια στήλη**.
- Η εσοχή καθορίζει σε ποιον «γονέα» ανήκει ένα κλειδί. Αν μετακινήσεις το `limits:` δύο κενά αριστερά, δεν ανήκει πια στο `resources:` αλλά στο container — και ο Kubernetes θα το απορρίψει.
- Σε στοιχείο λίστας, τα επόμενα κλειδιά του ίδιου αντικειμένου στοιχίζονται κάτω από το πρώτο κλειδί μετά την παύλα:

```
ports:
  - name: http          # η παύλα ξεκινά ένα νέο αντικείμενο της λίστας
    containerPort: 8000 # στοιχισμένο με το "name", άρα ανήκει στο ΙΔΙΟ αντικείμενο
```

Το ίδιο κομμάτι σε JSON, για σύγκριση:

```
{ "ports": [ { "name": "http", "containerPort": 8000 } ] }
```

3.3 Τύποι και παγίδες

Γράφεις	Το YAML το διαβάζει ως	Σχόλιο
`port: 8000`	αριθμός 8000	
`port: "8000"`	κείμενο "8000"	Τα εισαγωγικά επιβάλλουν string
`enabled: yes`	true (σε YAML 1.1) ή "yes"	Διφορούμενο· προτιμήστε true/false
`version: 1.10`	αριθμός 1.1 (!)	Βάλτε εισαγωγικά: "1.10"
`cpu: 100m`	κείμενο "100m"	Ο Kubernetes το ερμηνεύει ως 0,1 CPU
`country: NO`	false σε YAML 1.1	Το γνωστό «Norway problem»

3.4 Πολλαπλά έγγραφα, πολυγραμμικό κείμενο, anchors

```
# Τρεις παύλες χωρίζουν πολλά "έγγραφα" στο ίδιο αρχείο.
# Έτσι μπορείς να βάλεις Deployment και Service σε ΕΝΑ αρχείο:
apiVersion: apps/v1
kind: Deployment
...
---
apiVersion: v1
kind: Service
...

# Πολυγραμμικό κείμενο:
script: |            # "|" κρατάει τις αλλαγές γραμμής όπως είναι
  echo γραμμή 1
  echo γραμμή 2
περιγραφή: >         # ">" ενώνει τις γραμμές σε μία (με κενά)
  μεγάλο κείμενο
  σε πολλές γραμμές

# Anchors (&) και aliases (*): επαναχρησιμοποίηση τιμών
defaults: &res
  cpu: 100m
  memory: 128Mi
web:
  resources: *res     # ίδιο με το παραπάνω
```

Στο repository τα δύο manifests είναι σε ξεχωριστά αρχεία (`deployment.yaml`, `service.yaml`). Η εντολή `kubectl apply -f k8s/` διαβάζει όλα τα αρχεία του φακέλου, άρα το αποτέλεσμα είναι ίδιο με ένα αρχείο με `---`.

3.5 Πώς ελέγχεις ένα YAML

- `kubectl apply -f k8s/ --dry-run=client` – ελέγχει σύνταξη και σχήμα χωρίς να αλλάξει τίποτα στο cluster.
- `kubectl apply -f k8s/ --dry-run=server` – ο API server επικυρώνει πλήρως το αντικείμενο.
- `kubectl explain deployment.spec.template.spec.containers` – δείχνει ποια πεδία επιτρέπονται και τι σημαίνουν.
- `python -c "import yaml,sys; print(yaml.safe_load(open(sys.argv[1])))" αρχείο.yaml` – δείχνει πώς διαβάστηκε η δομή.
- Επεκτάσεις editor (π.χ. «YAML» της Red Hat στο VS Code) δίνουν αυτόματη συμπλήρωση και έλεγχο για Kubernetes.

4. Θεωρία: Docker

Το **Docker** πακετάρει μια εφαρμογή μαζί με ό,τι χρειάζεται για να τρέξει (λειτουργικό βάσης, Python, βιβλιοθήκες, κώδικα, δεδομένα) σε ένα **image**. Από το image ξεκινούν **containers**: απομονωμένες διεργασίες που μοιράζονται τον πυρήνα (kernel) του host αλλά έχουν το δικό τους σύστημα αρχείων, δίκτυο και διεργασίες. Έτσι «τρέχει παντού το ίδιο» – στο laptop, στον server, στον Kubernetes.

4.1 Βασικές έννοιες

Έννοια	Επεξήγηση
Image	Πρότυπο μόνο-για-ανάγνωση, φτιαγμένο από στρώματα (layers). Π.χ. `video-streaming:1.0`.
Container	Τρέχουσα «παρουσία» ενός image. Μπορούν να τρέχουν πολλά containers από το ίδιο image.
Dockerfile	Αρχείο κειμένου με τις οδηγίες κατασκευής του image, βήμα-βήμα.
Layer	Κάθε εντολή RUN/COPY δημιουργεί ένα νέο στρώμα. Τα στρώματα αποθηκεύονται στην cache και ξαναχρησιμοποιούνται.
Tag	Ετικέτα έκδοσης μετά την «:», π.χ. `:1.0`, `:1.1`. Χωρίς tag εννοείται `:latest`.
Registry	Αποθήκη images (Docker Hub, GHCR). Εδώ δεν χρησιμοποιείται: το image φορτώνεται απευθείας στο minikube.
Build context	Ο φάκελος που στέλνεται στο Docker κατά το build (εδώ η τελεία `.` = ρίζα του έργου).
Port mapping	`-p 8000:8000` = θύρα host : θύρα container.

4.2 Οι εντολές του Dockerfile

Εντολή	Τι κάνει
`FROM`	Το image βάσης από το οποίο ξεκινάμε. Πάντα πρώτη εντολή.
`ENV`	Ορίζει μεταβλητές περιβάλλοντος που ισχύουν στο build και στο container.
`WORKDIR`	Ο τρέχων φάκελος για τις επόμενες εντολές (τον δημιουργεί αν δεν υπάρχει).
`COPY src dst`	Αντιγράφει αρχεία από το build context μέσα στο image.
`RUN`	Εκτελεί εντολή *κατά το build* (π.χ. pip install) και αποθηκεύει το αποτέλεσμα ως layer.
`USER`	Με ποιον χρήστη τρέχουν οι επόμενες εντολές και το container.
`EXPOSE`	Τεκμηρίωση της θύρας που ακούει η εφαρμογή (δεν την ανοίγει μόνη της).
`CMD`	Η εντολή που τρέχει όταν *ξεκινά* το container. Μία ανά Dockerfile.
`ENTRYPOINT / ARG / VOLUME`	Άλλες εντολές (δεν χρησιμοποιούνται εδώ): σταθερό εκτελέσιμο, μεταβλητές build, σημεία volume.

4.3 Build cache – γιατί έχει σημασία η σειρά

Το Docker ξαναχτίζει ένα layer μόνο αν άλλαξε η εντολή ή τα αρχεία που αντιγράφει — και τότε ξαναχτίζει **και όλα τα επόμενα**. Γι' αυτό το Dockerfile του έργου αντιγράφει **πρώτα μόνο το requirements.txt** και τρέχει pip install, και **μετά** τον κώδικα. Όταν αλλάζεις μόνο κώδικα Python, η εγκατάσταση βιβλιοθηκών έρχεται από την cache και το build παίρνει

δευτερόλεπτα.

4.4 Πώς φτιάχνεις ένα Dockerfile από το μηδέν (συνταγή)

- **1. Διάλεξε βάση:** το επίσημο image της γλώσσας, κατά προτίμηση «slim» (μικρότερο, λιγότερα κενά ασφαλείας).
- **2. Όρισε WORKDIR** ώστε όλες οι διαδρομές να είναι σχετικές με αυτόν.
- **3. Αντέγραψε πρώτα τα αρχεία εξαρτήσεων** (requirements.txt, package.json) και εγκατέστησέ τα.
- **4. Αντέγραψε τον κώδικα** και τα δεδομένα.
- **5. Γίνε μη-root χρήστης** για ασφάλεια.
- **6. EXPOSE + CMD:** η εφαρμογή πρέπει να ακούει στο `0.0.0.0`, όχι στο `127.0.0.1`, αλλιώς δεν είναι προσβάσιμη έξω από το container.
- **7. Γράψε .dockerignore** για να μη στέλνονται άχρηστα/μυστικά αρχεία στο build.

```
docker build -t video-streaming:1.0 .              # χτίζει το image από το Dockerfile της τρέχουσας ρίζας
docker images                                      # λίστα images
docker run --rm -p 8000:8000 --name vs video-streaming:1.0   # τρέχει container
docker ps                                          # τρέχοντα containers
docker logs vs                                     # logs του container
docker exec -it vs sh                              # shell μέσα στο container
docker stop vs                                     # σταματά (το --rm το διαγράφει αυτόματα)
```

5. Θεωρία: Kubernetes

Το Docker τρέχει containers σε *ένα* μηχάνημα. Το **Kubernetes (K8s)** είναι ένας **orchestrator**: διαχειρίζεται containers σε ένα **cluster** πολλών μηχανημάτων. Του λες *τι θέλεις* (π.χ. «3 αντίγραφα αυτού του image, με 0,1 CPU το καθένα, προσβάσιμα στη θύρα 30080») και αυτό φροντίζει συνεχώς να ισχύει: τοποθετεί τα containers στους κόμβους, τα επανεκκινεί αν πέσουν, τα αντικαθιστά αν χαθεί κόμβος και μοιράζει την κίνηση.

5.1 Δηλωτικό μοντέλο (declarative) και reconciliation loop

Στον Kubernetes δεν δίνεις εντολές του τύπου «ξεκίνα container». Γράφεις την **επιθυμητή κατάσταση** (desired state) σε αρχεία YAML (τα λεγόμενα **manifests**) και τα στέλνεις με `kubectl apply`. Οι **controllers** του Kubernetes συγκρίνουν συνεχώς την επιθυμητή με την **πραγματική κατάσταση** και κάνουν ό,τι χρειάζεται για να ταυτιστούν. Αν σβήσεις με το χέρι ένα Pod, ο Deployment controller βλέπει 2 αντί για 3 και φτιάχνει αμέσως νέο.

5.2 Τα βασικά αντικείμενα

Αντικείμενο	Ρόλος
Cluster	Το σύνολο: control plane (API server, scheduler, controllers, etcd) + worker nodes.
Node	Ένα μηχάνημα (φυσικό ή εικονικό) που τρέχει Pods. Στο minikube κάθε node είναι ένα container Docker.
Pod	Η μικρότερη μονάδα: ένα ή περισσότερα containers που μοιράζονται IP και δίκτυο. Είναι «αναλώσιμο».
Deployment	Κρατά N πανομοιότυπα Pods από ένα template· κάνει rolling updates και rollbacks.
ReplicaSet	Δημιουργείται αυτόματα από το Deployment· αυτό μετρά και διατηρεί τα Pods.
Service	Σταθερή διεύθυνση (IP + DNS όνομα) μπροστά σε ένα μεταβαλλόμενο σύνολο Pods· κάνει load balancing.
Labels / Selectors	Ετικέτες κλειδί=τιμή. Το Deployment και το Service βρίσκουν «τα δικά τους» Pods μέσω selector.
Probes	Έλεγχοι υγείας: readiness (να δέχεται κίνηση;), liveness (να επανεκκινηθεί;), startup.
HPA	Horizontal Pod Autoscaler: αλλάζει αυτόματα τα replicas με βάση π.χ. τη χρήση CPU (βήμα 5 του έργου).
Namespace	Λογικός διαχωρισμός αντικειμένων. Εδώ όλα μπαίνουν στο `default`.

5.3 Η ανατομία κάθε manifest Kubernetes

Κάθε αρχείο YAML του Kubernetes έχει **τα ίδια τέσσερα κλειδιά πρώτου επιπέδου**:

```
apiVersion: apps/v1        # 1. Ποια ομάδα/έκδοση του API ορίζει αυτό το είδος αντικειμένου
kind: Deployment           # 2. Τι είδους αντικείμενο είναι
metadata:                  # 3. Ταυτότητα: όνομα, namespace, labels, annotations
  name: video-streaming
  labels:
    app: video-streaming
spec:                      # 4. Η ΕΠΙΘΥΜΗΤΗ κατάσταση - διαφέρει ανά kind
  replicas: 3
  ...
# (υπάρχει και 5ο, το status, αλλά το γράφει ο Kubernetes, όχι εσύ)
```

kind	apiVersion
Pod, Service, ConfigMap, Secret, Namespace	`v1` (core group)
Deployment, ReplicaSet, StatefulSet, DaemonSet	`apps/v1`
HorizontalPodAutoscaler	`autoscaling/v2`
Ingress	`networking.k8s.io/v1`
Job, CronJob	`batch/v1`

Δεν χρειάζεται να τα θυμάσαι: η εντολή `kubectl api-resources` τα δείχνει όλα.

5.4 Πώς φτιάχνεις τα YAML του Kubernetes από το μηδέν (συνταγή)

Βήμα 1 – Ζήτα από το kubectl έναν σκελετό αντί να γράφεις από το μηδέν:

```
kubectl create deployment video-streaming --image=video-streaming:1.0 --replicas=3 \
        --port=8000 --dry-run=client -o yaml > k8s/deployment.yaml

kubectl create service nodeport video-streaming --tcp=80:8000 \
        --dry-run=client -o yaml > k8s/service.yaml
```

Το `--dry-run=client -o yaml` σημαίνει «μη δημιουργήσεις τίποτα, απλώς τύπωσε το YAML που θα έστελνες». Μετά καθαρίζεις κενά πεδία (π.χ. `creationTimestamp: null`, `status: {}`) και προσθέτεις ό,τι λείπει.

Βήμα 2 – Συμπλήρωσε το Deployment με τα σημαντικά πεδία, ένα-ένα:

- **labels + selector:** το `spec.selector.matchLabels` ΠΡΕΠΕΙ να ταιριάζει με τα `spec.template.metadata.labels`, αλλιώς απορρίπτεται.
- **image + imagePullPolicy:** για τοπικό image στο minikube βάλε `IfNotPresent` (ή `Never`).
- **ports:** η θύρα που ακούει η εφαρμογή στο container (`containerPort: 8000`), προαιρετικά με όνομα.
- **resources:** requests (τι εγγυάται/δεσμεύει ο scheduler) και limits (το ανώτατο όριο).
- **readinessProbe / livenessProbe:** τα endpoints υγείας της εφαρμογής.
- **Τοποθέτηση:** topologySpreadConstraints, affinity ή nodeSelector για το σε ποιους κόμβους πάνε τα Pods.

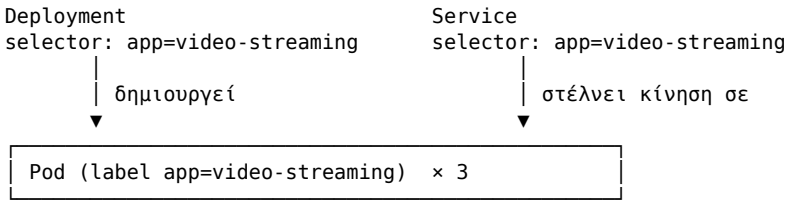
Βήμα 3 – Συμπλήρωσε το Service: `selector` ίδιο με τα labels των Pods, `port` (θύρα του Service), `targetPort` (θύρα του Pod), και `type`:

type	Πού είναι προσβάσιμο
`ClusterIP` (προεπιλογή)	Μόνο μέσα στο cluster, στο DNS όνομα του Service.
`NodePort`	Επιπλέον σε μια σταθερή θύρα (30000–32767) σε *κάθε* κόμβο. Χρησιμοποιείται εδώ.
`LoadBalancer`	Επιπλέον εξωτερικός load balancer του cloud provider (ή minikube tunnel).

Βήμα 4 – Έλεγξε, εφάρμοσε, παρακολούθησε:

```
kubectl apply -f k8s/ --dry-run=server     # επικύρωση
kubectl apply -f k8s/                      # δημιουργία / ενημέρωση (idempotent: τρέχει όσες φορές θες)
kubectl get pods -o wide                   # ποια Pods, σε ποιους κόμβους, αν είναι READY
kubectl describe pod <όνομα>               # events: γιατί δεν ξεκινά ένα Pod
kubectl get endpoints video-streaming      # ποιες IP Pods έχει πίσω του το Service
kubectl delete -f k8s/                     # αφαίρεση
```

5.5 Πώς ταιριάζουν όλα μαζί: labels



Ούτε το Deployment ούτε το Service «ξέρουν» ονόματα Pods. Βρίσκουν τα Pods μόνο μέσω της ετικέτας `app: video-streaming`. Αν ένα Pod πεθάνει και δημιουργηθεί καινούργιο με άλλο όνομα και IP, το Service το βρίσκει αυτόματα — αυτό είναι το **service discovery**.

6. Επεξήγηση κάθε αρχείου

Σε αυτή την ενότητα κάθε αρχείο του repository εξηγείται με τη σειρά που εκτελείται στην πράξη: πρώτα η προετοιμασία βίντεο, μετά ο κώδικας Python, η σελίδα demo, το Docker, ο Kubernetes και τέλος τα βοηθητικά αρχεία.

6.1 scripts/prepare_media.py – μετατροπή βίντεο σε HLS

Ρόλος: εργαλείο γραμμής εντολών που τρέχει *μία φορά, πριν το docker build*. Παίρνει ένα κανονικό βίντεο (π.χ. MP4) και καλεί το πρόγραμμα **ffmpeg** για να το κόψει σε segments HLS μέσα στο `media/hls/<id>/`. Αν δεν υπάρχει βίντεο, μπορεί να φτιάξει ένα συνθετικό test pattern.

Πώς λειτουργεί, βήμα-βήμα

- **Σταθερές διαδρομών:** `PROJECT_ROOT = Path(__file__).resolve().parent.parent` – το script είναι στο `scripts/`, οπότε ανεβαίνει ένα επίπεδο. Από εκεί ορίζει `SOURCE_DIR` και `HLS_DIR`.
- **VIDEO_ID_PATTERN:** ίδια κανονική έκφραση με το `video_service.py` (μόνο γράμματα, ψηφία, _ και -). Έτσι το script δεν φτιάχνει ποτέ βίντεο που ο server θα αρνιόταν να σερβίρει.
- **parse_args():** με το `argparse` διαβάζει τις επιλογές `--input`, `--id`, `--segment` (προεπιλογή 4 δευτ.), `--test` και `--duration`. Το argparse φτιάχνει αυτόματα και το `--help`.
- **input_options():** επιστρέφει τα ορίσματα εισόδου του ffmpeg. Σε λειτουργία test χρησιμοποιεί τη «συσκευή» `lavfi` του ffmpeg με `testsrc` (χρωματιστές μπάρες με χρονόμετρο, 1280x720, 25fps) και `sine` (τόνος 440 Hz) ως ήχο. Αλλιώς ελέγχει ότι το αρχείο υπάρχει και επιστρέφει `-i αρχείο`.
- **main():** (α) ελέγχει με `shutil.which` ότι υπάρχει ffmpeg, (β) διαλέγει id (από `--id`, «test», ή το όνομα αρχείου χωρίς κατάληξη μέσω `.stem`), (γ) δημιουργεί τον φάκελο εξόδου και σβήνει παλιά `.ts` ώστε να μη μείνουν ορφανά segments, (δ) χτίζει την εντολή ffmpeg ως **λίστα** (ασφαλές με κενά στις διαδρομές, χωρίς shell injection), (ε) την τρέχει με `subprocess.run`, (στ) τυπώνει σύνοψη: πλήθος segments, MB, URL.
- `if __name__ == "__main__":` – η `main()` τρέχει μόνο όταν εκτελείται το αρχείο, όχι όταν γίνεται import.

Οι παράμετροι του ffmpeg

Παράμετρος	Σημασία
`-y`	Αντικατάσταση υπαρχόντων αρχείων χωρίς ερώτηση
`-hide_banner -loglevel warning -stats`	Λιγότερη έξοδος· μόνο προειδοποιήσεις και πρόοδος
`-c:v libx264`	Κωδικοποίηση βίντεο H.264, υποστηρίζεται από όλους τους browsers
`-preset veryfast`	Ταχύτητα κωδικοποίησης έναντι μεγέθους αρχείου
`-crf 23`	Ποιότητα (μικρότερο = καλύτερο/μεγαλύτερο αρχείο)
`-pix_fmt yuv420p`	Μορφή pixel συμβατή με κάθε player
`-force_key_frames expr:gte(t,n_forced*4)`	Keyframe ακριβώς κάθε 4 δευτ. ώστε κάθε segment να ξεκινά από keyframe και να έχει ίδια διάρκεια
`-c:a aac -b:a 128k`	Ήχος AAC στα 128 kbps

Παράμετρος	Σημασία
`-shortest`	Σταματά στο τέλος της πιο σύντομης εισόδου (χρειάζεται στο test pattern + τόνο)
`-f hls -hls_time 4`	Έξοδος HLS με segments ~4 δευτ.
`-hls_playlist_type vod`	Πλήρης playlist με #EXT-X-ENDLIST
`-hls_segment_filename seg_%03d.ts`	Ονόματα seg_000.ts, seg_001.ts ... (το %03d = αριθμός 3 ψηφίων)

```
python scripts/prepare_media.py                                    # media/source/sample.mp4 -> media/hls/
  sample/
python scripts/prepare_media.py --input media/source/earth_spinning.mp4 --id earth
python scripts/prepare_media.py --test --duration 60               # test pattern -> media/hls/test/
```

6.2 backend/requirements.txt – εξαρτήσεις Python

```
fastapi==0.143.3
uvicorn[standard]==0.54.0
pydantic-settings==2.16.0
prometheus-client==0.26.0
```

Πακέτο	Γιατί χρειάζεται
fastapi	Το web framework: routing, validation, αυτόματα docs (/docs)
uvicorn[standard]	Ο ASGI server που τρέχει την εφαρμογή FastAPI και ακούει στη θύρα 8000. Το [standard] φέρνει γρηγορότερα extras (uvloop, httptools)
pydantic-settings	Διαβάζει ρυθμίσεις από μεταβλητές περιβάλλοντος / .env σε μια typed κλάση (config.py)
prometheus-client	Ορίζει μετρικές (Counter, Gauge, Histogram) και τις εξάγει σε μορφή Prometheus

Οι εκδόσεις είναι «καρφωμένες» με `==` ώστε κάθε build να βγάζει ακριβώς το ίδιο αποτέλεσμα (reproducible builds).

6.3 backend/app/core/config.py – ρυθμίσεις

Ρόλος: κεντρικό σημείο για όλες τις ρυθμίσεις. Ακολουθεί την αρχή **12-factor**: η ρύθμιση έρχεται από μεταβλητές περιβάλλοντος, οπότε *το ίδιο image* τρέχει αλλαγμένο μόνο μέσω env vars σε laptop, Docker και Kubernetes.

```
# Gives access to hostname
import socket;

# Path: Handles paths
from pathlib import Path

# BaseSettings: Class that reads values from env vars
# SettingsConfigDict: Its settings
from pydantic_settings import BaseSettings, SettingsConfigDict

# Sets project root by climbing 4 levels
PROJECT_ROOT = Path(__file__).resolve().parents[3]

# Each field of this class is a setting that gets autofilled my base settings
class Settings(BaseSettings):
    # Settings on how the values are read
    model_config = SettingsConfigDict(
        # Reads from file .env if exists
        env_file=".env",
        # env file encoding
        env_file_encoding="utf-8",
        # Ignore values that are note mentioned here
```

```
        extra="ignore"
    )

    # App name. Env var = APP_NAME
    app_name: str = "Video Streaming Service"
    #Folder with videos
    media_dir: Path = PROJECT_ROOT / "media"
    # Machine name
    instance_name: str = socket.gethostname()

    @property
    # Folder with hls files
    def hls_dir(self) -> Path:
        return self.media_dir / "hls"

# Created one time
settings = Settings()
```

- `PROJECT_ROOT = Path(__file__).resolve().parents[3]`: το αρχείο είναι στο `<root>/backend/app/core/config.py`. `parents[0]`=core, [1]=app, [2]=backend, [3]=ρίζα έργου.
- **class Settings(BaseSettings):** κάθε πεδίο της κλάσης είναι μια ρύθμιση. Το pydantic-settings ψάχνει αυτόματα μεταβλητή περιβάλλοντος με το ίδιο όνομα (χωρίς διάκριση πεζών/κεφαλαίων): `app_name` ← `APP_NAME`, `media_dir` ← `MEDIA_DIR`, `instance_name` ← `INSTANCE_NAME`. Αν δεν βρει, κρατά την προεπιλογή. Μετατρέπει και τον τύπο (π.χ. string σε `Path`).
- **model_config:** διαβάζει επίσης αρχείο `.env` αν υπάρχει, σε UTF-8, και `extra="ignore"` αγνοεί άγνωστες μεταβλητές.
- **instance_name = socket.gethostname()**: μέσα σε container το hostname είναι το ID του container· μέσα σε Kubernetes είναι **το όνομα του Pod**. Έτσι κάθε απάντηση δείχνει ποιο αντίγραφο την εξυπηρέτησε.
- **@property hls_dir:** υπολογιζόμενη τιμή `media_dir / "hls"` (δεν είναι ξεχωριστή ρύθμιση).
- **settings = Settings()**: δημιουργείται *μία φορά* κατά το import· όλα τα modules εισάγουν αυτό το ίδιο αντικείμενο.
- Στο Docker το `MEDIA_DIR=/app/media` ορίζεται στο Dockerfile, γιατί εκεί η δομή φακέλων είναι διαφορετική (δεν υπάρχει φάκελος backend/).

> Μικρή λεπτομέρεια: το `import socket;` έχει περιττό ερωτηματικό — στην Python είναι έγκυρο αλλά δεν χρειάζεται.

6.4 backend/app/services/video_service.py – λογική εύρεσης βίντεο

Ρόλος: το «επίπεδο υπηρεσίας» (service layer). Περιέχει τη λογική που δεν σχετίζεται με HTTP: ποια βίντεο υπάρχουν και πώς βρίσκεται με **ασφάλεια** ένα αρχείο στον δίσκο. Τα routes (videos.py, health.py) το καλούν.

```
# Regular expressions
import re

#Path
from pathlib import Path

# Settings we take from hls_dir
from app.core.config import settings

# Playlist name that ffmpeg creatres in each video folder
```

```
PLAYLIST_NAME = "index.m3u8"

# Valid video id letters numbers and _
VIDEO_ID_PATTERN = re.compile(r"^[A-Za-z0-9_-]+$")

# Valid filename
FILENAME_PATTERN = re.compile(r"^[A-Za-z0-9_-]+\.(m3u8|ts)$")

# What video types are allowed
MEDIA_TYPES = {
    # Official HLS type
    ".m3u8": "application/vnd.apple.mpegurl",
    # Officail MPEG-TS
    ".ts": "video/mp2t"
}

# Video check to see if exists or found
class VideoNotFoundError(Exception):
    pass

# Rerurns names of avaialvle videos
def list_videos() -> list[str]:
    # If folder not exists retur empty list
    hls_dir = settings.hls_dir
    if not hls_dir.is_dir():
        return []
    # Rerturns sorted video names
    return sorted(
        # Subfolder name
        d.name
        # For each folder
        for d in hls_dir.iterdir()
        # Keep only folder that contain the playlist name
        if(d / PLAYLIST_NAME).is_file()
    )

# Checks to see if at least one video exists
def media_available() -> bool:
    return len(list_videos()) > 0

#Find specified video file
def resolve_video_file(video_id: str, filename: str) -> tuple[Path, str]:
    # Check tro see if video has allowed form
    if not VIDEO_ID_PATTERN.fullmatch(video_id):
        raise VideoNotFoundError(video_id)

    # Check to see if video has allowed name
    if not FILENAME_PATTERN.fullmatch(filename):
        raise VideoNotFoundError(filename)

    # Get realitive path
    base = settings.hls_dir.resolve()
    path = (base / video_id / filename).resolve()

    # Check to see if file actually is isnide folder
    if not path.is_relative_to(base):
        raise VideoNotFoundError(filename)
    # Check to see if file exists in disc
    if not path.is_file():
        raise VideoNotFoundError(filename)

    # Rerturn path and type
    return path, MEDIA_TYPES[path.suffix]
```

Συναρτήσεις

- **list_videos()**: αν δεν υπάρχει ο φάκελος hls επιστρέφει κενή λίστα. Αλλιώς διατρέχει τους υποφακέλους με `iterdir()` και κρατά μόνο όσους περιέχουν `index.m3u8` (ένα έτοιμο βίντεο). Επιστρέφει τα ονόματα ταξινομημένα. Είναι γραμμένη ως generator expression μέσα στο `sorted()`.

- **media_available()**: `True` αν υπάρχει τουλάχιστον ένα βίντεο. Τη χρησιμοποιεί το readiness probe.
- **resolve_video_file(video_id, filename)**: μετατρέπει τα κομμάτια του URL σε πραγματική διαδρομή αρχείου και επιστρέφει (διαδρομή, Content-Type). Αν κάτι δεν πάει καλά, πετά `VideoNotFoundError`.

Η προστασία από path traversal (σε τρία επίπεδα)

Ο κίνδυνος: ένας επιτιθέμενος θα μπορούσε να ζητήσει κάτι σαν `/videos/../../etc/passwd` για να διαβάσει αρχεία έξω από τον φάκελο βίντεο. Ο κώδικας το εμποδίζει με «άμυνα σε βάθος»:

- **1. Whitelist ονομάτων:** `VIDEO_ID_PATTERN` επιτρέπει μόνο `[A-Za-z0-9_-]`, και `FILENAME_PATTERN` μόνο ονόματα που τελειώνουν σε `.m3u8` ή `.ts`. Τελείες, κάθετοι και `..` απορρίπτονται αμέσως. Το `fullmatch` απαιτεί να ταιριάζει ολόκληρο το string.
- **2. Έλεγχος περιορισμού:** `.resolve()` μετατρέπει τη διαδρομή σε απόλυτη (λύνοντας symlinks και ..) και το `is_relative_to(base)` επιβεβαιώνει ότι παραμένει μέσα στο `media/hls`.
- **3. Ύπαρξη:** `is_file()` – πρέπει να είναι υπαρκτό αρχείο, όχι φάκελος.
- **MEDIA_TYPES:** αντιστοιχεί κατάληξη σε σωστό Content-Type: `application/vnd.apple.mpegurl` για playlist και `video/mp2t` για segments. Οι players χρειάζονται τον σωστό τύπο.

Η δική μας εξαίρεση `VideoNotFoundError` κρατά το service ανεξάρτητο από το HTTP· το route τη μεταφράζει σε 404.

6.5 backend/app/api/routes/videos.py – endpoints βίντεο

```
# Api router: groups endpoints
# HTTPException: Throws HTTP errors
from fastapi import APIRouter, HTTPException

#FileResponse: sends a file from disc in chunks
from fastapi.responses import FileResponse

# Settings
from app.core.config import settings

# Service thay finds video files
from app.services import video_service

# Video not found exception
from app.services.video_service import VideoNotFoundError

# endpoint prefix
router = APIRouter(prefix="/videos", tags=["videos"])

# GET /videos
@router.get("")
# Returns list of videos
def list_videos() -> dict:
    # Ask the service which video sexist
    video_ids = video_service.list_videos()
    return {
        # For every video get its id and the url of its playlist
        "videos": [{"id": v, "playlist": f"/videos/{v}/index.m3u8"} for v in video_ids],
        "instance": settings.instance_name
    }

# GET /videos/[video_id]/{filename}
@router.get("/{video_id}/{filename}")
```

```
# Fastapi filles video id and filenme automatically
def get_video_file(video_id: str, filename: str) -> FileResponse:
    try:
        path, media_type = video_service.resolve_video_file(video_id, filename)
    except VideoNotFoundError:
        # raise 404 error if not found
        raise HTTPException(status_code=404, detail="Video file not found") from None

    # Send the file
    return FileResponse(
        # Path of the file on disk
        path,
        # Content-Type
        media_type=media_type,
        # Header with instance name
        headers={"X-Served-By": settings.instance_name}
    )
```

- **APIRouter(prefix="/videos", tags=["videos"])**: ομάδα endpoints που όλα ξεκινούν με /videos. Το tag τα ομαδοποιεί στη σελίδα /docs.
- **GET /videos** (`@router.get("")`): ρωτά το service ποια βίντεο υπάρχουν και επιστρέφει dict, που το FastAPI μετατρέπει αυτόματα σε JSON, π.χ. `{"videos":[{"id":"sample","playlist":"/videos/sample/index.m3u8"}],"instance":"video-streaming-7d9c-abcde"}`.
- **GET /videos/{video_id}/{filename}**: τα `{...}` είναι *path parameters*· το FastAPI τα περνά ως ορίσματα στη συνάρτηση. Καλεί `resolve_video_file`. Σε `VideoNotFoundError` πετά `HTTPException(404)`. Το `from None` κρύβει το αρχικό traceback (καθαρότερα logs).
- **FileResponse:** στέλνει το αρχείο από τον δίσκο σε κομμάτια (streaming), χωρίς να το φορτώσει όλο στη μνήμη· βάζει αυτόματα Content-Length, Last-Modified και ETag.
- **X-Served-By header:** προσαρμοσμένο header με το όνομα του Pod. Η σελίδα demo το διαβάζει για να δείξει το load balancing.
- Οι συναρτήσεις είναι απλές `def` (όχι async): το FastAPI τις τρέχει σε threadpool, οπότε η πρόσβαση στον δίσκο δεν μπλοκάρει τον server.

6.6 backend/app/api/routes/health.py – έλεγχοι υγείας

```
# APIRouter: a group of endpoints that is plugged into the app with include_router (see main.py)
from fastapi import APIRouter

# JSONResponse: a JSON response where we choose the status code ourselves (e.g. 503)
from fastapi.responses import JSONResponse

# Settings, for the instance name
from app.core.config import settings

# The service, to check whether any videos exist
from app.services import video_service

# prefix: every endpoint here starts with /health. tags: groups them on the /docs page
router = APIRouter(prefix="/health", tags=["health"])


# GET /health/live
@router.get("/live")
# Liveness: if the app can run this function at all, it is alive
def live() -> dict:
    # FastAPI turns this into JSON with status 200. "instance" shows which Pod answered
    return {"status": "alive", "instance": settings.instance_name}


# GET /health/ready
@router.get("/ready")
# Readiness: can it actually serve videos?
def ready() -> JSONResponse:
```

```
    # Is there at least one ready video (a folder with index.m3u8)?
    if video_service.media_available():
        # Yes -> ready
        return JSONResponse(
            # Response body
            {"status": "ready", "instance": settings.instance_name},
            # 200 OK: Kubernetes will send traffic to this Pod
            status_code=200,
        )
    # No -> not ready
    return JSONResponse(
        # Explain the reason, for debugging
        {"status": "not ready", "reason": "no videos found", "instance": settings.instance_name},
        # 503 Service Unavailable: Kubernetes will stop sending traffic here
        status_code=503,
    )
```

Δύο διαφορετικές ερωτήσεις που κάνει ο Kubernetes:

	/health/live (liveness)	/health/ready (readiness)
Ερώτηση	«Ζει η διεργασία;»	«Μπορεί να εξυπηρετήσει βίντεο τώρα;»
Λογική	Αν η συνάρτηση τρέχει, η εφαρμογή ζει → πάντα 200	200 αν `media_available()`, αλλιώς 503 Service Unavailable
Αν αποτύχει 3 φορές	Ο Kubernetes **επανεκκινεί** το container	Ο Kubernetes **βγάζει το Pod από το Service** (δεν του στέλνει κίνηση), χωρίς επανεκκίνηση
Γιατί έτσι	Πιάνει κολλημένη διεργασία (deadlock)	Ένα image χωρίς βίντεο δεν πρέπει να δέχεται θεατές

`JSONResponse` χρησιμοποιείται στο ready γιατί θέλουμε να ορίσουμε εμείς το status code (503)· στο live αρκεί ένα dict (200 εξ ορισμού).

6.7 backend/app/core/metrics.py – μετρικές Prometheus

Ρόλος: μετρά κάθε αίτημα HTTP ώστε να αξιολογηθεί η απόδοση (βήμα 6 του έργου). Το **Prometheus** είναι σύστημα παρακολούθησης που κάθε λίγα δευτερόλεπτα «τραβά» (scrape) το `/metrics` και αποθηκεύει τις τιμές ως χρονοσειρές.

```
# Time: to claculate time
import time

# Request: it is what contained all the inforamtion of the incoming request
from fastapi import Request

# Base metrics type
from prometheus_client import Counter, Gauge, Histogram

# What are metrics
# Counter: counts the total amount of request for example only goes up
# Gauge: Value that gors up and down used for example to calcualte how many requests
# are bbeing handled now
# Histogram: sorts values into "bucekts", so we can get %

# TRequest Counter
REQUESTS_TOTAL = Counter(
    # Metric name
    "http_requests_total",
    # Metric description
    "Total amount of HTTP requests",
    # Labels: divides metrics based on method, endpoint and status
    ["method", "path", "status"]
)

# Count time to fully get a request
REQUEST_DURATION = Histogram(
```

```
    "http_request_duration_seconds",
    "Total amount of HTTP requests",
    ["method", "path"],
    # Bucket limit
    buckets=(0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0, 10.0)
)

# Requests currently dealing with
REQUESTS_IN_PROGRESS = Gauge(
    "http_requests_in_progress",
    "Total HTTP requests currently servicing"
)

# Total amount of bytes sent
BYTES_SENT_TOTAL = Counter(
    "http_response_bytes_total",
    "Total bytes send to clients",
    ["path"]
)

# Runs in every request
async def metrics_middleware(request: Request, call_next):
    # Not counting requests for metrics themselves
    if request.url.path == "/metrics":
        # Forward to not sque the metrics
        return await call_next(request)

    # Time of start
    start = time.perf_counter()
    # Increase requests servicing by one
    REQUESTS_IN_PROGRESS.inc()

    try:
        # Get endpoint
        response = await call_next(request)
    except Exception:
        REQUESTS_IN_PROGRESS.dec()
        # Raise throws the error back to fastapi to handle it tghere
        raise

    # Gets route that got matached
    route = request.scope.get("route")

    path = route.path if route else "unmatched"
    REQUESTS_TOTAL.labels(
        request.method,
        path,
        str(response.status_code)).inc()

    # Response body is sent in pieces
    original_body = response.body_iterator

    async def tracked_body():
        # Counts bytes
        sent = 0
        # Try fianlly ensures the metrics are counterd even if the client cuts
        try:
            # For every bit of data from file
            async for chunk in original_body:
                # Count bytes
                sent += len(chunk)
                # Send it to client as is
                yield chunk
        # Acts when the sent is done either succesfully or not
        finally:
            # Count how many bytes were send
            BYTES_SENT_TOTAL.labels(path).inc(sent)

            # Count total time (now - start)
            REQUEST_DURATION.labels(
                request.method,
                path).observe(time.perf_counter() - start)
```

```
            # Remove after finish active request
            REQUESTS_IN_PROGRESS.dec()

            # Replace defualt repsonse body with our
        response.body_iterator = tracked_body()
        return response
```

Οι τέσσερις μετρικές

Μετρική	Τύπος	Labels	Τι μετρά
`http_requests_total`	Counter	method, path, status	Σύνολο αιτημάτων (μόνο ανεβαίνει). Από τον ρυθμό αύξησης βγαίνουν τα αιτήματα/δευτ.
`http_request_duration_seconds`	Histogram	method, path	Χρόνος μέχρι να σταλεί και το τελευταίο byte. Τα buckets επιτρέπουν percentiles (p50, p95)
`http_requests_in_progress`	Gauge	—	Ταυτόχρονες μεταφορές τώρα (ανεβαίνει και κατεβαίνει)
`http_response_bytes_total`	Counter	path	Bytes που στάλθηκαν → throughput (MB/s)

Πώς δουλεύει το middleware

- Ένα **middleware** είναι συνάρτηση που τυλίγει *κάθε* αίτημα: τρέχει πριν και μετά το endpoint. Το `call_next(request)` προωθεί το αίτημα στο endpoint.
- Τα αιτήματα στο ίδιο το `/metrics` δεν μετριούνται, για να μη «λερώνουν» τα αποτελέσματα.
- Ξεκινά χρονόμετρο με `time.perf_counter()` (ακριβές μονοτονικό ρολόι) και αυξάνει το gauge.
- Αν το endpoint πετάξει εξαίρεση, μειώνει το gauge και την ξαναπετά (`raise`) για να τη χειριστεί το FastAPI.
- **Label path = το πρότυπο της διαδρομής** (`request.scope["route"].path`), π.χ. `/videos/{video_id}/{filename}` και όχι το πραγματικό URL. Αλλιώς κάθε segment θα έφτιαχνε νέα χρονοσειρά (cardinality explosion). Αιτήματα που δεν ταίριαξαν σε route γίνονται `unmatched`.
- **Το κόλπο με το body_iterator:** όταν επιστρέφει το `call_next`, έχουν σταλεί μόνο τα headers· το σώμα (π.χ. ένα segment 500 KB) στέλνεται μετά, σε κομμάτια. Γι' αυτό ο κώδικας αντικαθιστά τον iterator του σώματος με τον δικό του generator `tracked_body()`, που περνά κάθε chunk αυτούσιο στον client μετρώντας τα bytes.
- Το `finally` εξασφαλίζει ότι bytes, διάρκεια και μείωση του gauge καταγράφονται **όταν τελειώσει πραγματικά η μεταφορά**, ακόμη κι αν ο client κόψει τη σύνδεση.

> Λεπτομέρεια: η περιγραφή (help text) του Histogram λέει «Total amount of HTTP requests», αντιγραμμένη από τον Counter· θα ήταν ακριβέστερο κάτι σαν «HTTP request duration in seconds». Δεν επηρεάζει τη λειτουργία.

6.8 backend/app/main.py – το σημείο εισόδου της εφαρμογής

```
# Manages paths for both linux and windows
from pathlib import Path

# Class who create the web app
from fastapi import FastAPI

# FileResponse: Sends file from disc, Response: general answer
```

```
from fastapi.responses import FileResponse, Response

# Serves all files in a folder
from fastapi.staticfiles import StaticFiles
# Get routes used
from app.api.routes import videos, health

# Get settings used for the service
from app.core.config import settings

# Middleware that gets metrics for each request
from app.core.metrics import metrics_middleware

from prometheus_client import CONTENT_TYPE_LATEST, generate_latest # Tools to exctract metrics

STATIC_DIR = Path(__file__).parent / "static" # Static folder  where index.html is located

app = FastAPI(title=settings.app_name) # Creates app

# Metric middleware runs on each http request
app.middleware("http")(metrics_middleware)

# Adds endpoints /health
app.include_router(health.router)

#Adds endpoints to /videos
app.include_router(videos.router)

# Servers the files of app.static under URL /static
app.mount("/static", StaticFiles(directory=STATIC_DIR), name="static")

# Endpoint that prometheus reads
@app.get("/metrics", include_in_schema=False)
# Functions that runs when someones ask for GET /metrics
def metrics() -> Response:
        #  Returns metrics as txt
        return Response(generate_latest(), media_type=CONTENT_TYPE_LATEST)


# Landing page for browser demo
@app.get("/", include_in_schema=False)
def index() -> FileResponse:
        return FileResponse(STATIC_DIR / "index.html")
```

- `app = FastAPI(title=settings.app_name)`: δημιουργεί την εφαρμογή· ο τίτλος εμφανίζεται στο /docs.
- `app.middleware("http")(metrics_middleware)`: καταχωρεί το middleware μετρικών ώστε να τρέχει σε κάθε αίτημα HTTP (ισοδύναμο με decorator).
- `include_router(health.router)` και `include_router(videos.router)`: «κουμπώνει» τις ομάδες endpoints.
- `app.mount("/static", StaticFiles(...))`: σερβίρει τα αρχεία του φακέλου static/ κάτω από το URL /static (CSS, JS).
- **GET /metrics**: το `generate_latest()` παράγει όλες τις μετρικές σε μορφή κειμένου Prometheus· `CONTENT_TYPE_LATEST` είναι ο σωστός τύπος περιεχομένου. Το `include_in_schema=False` το κρύβει από το /docs.
- **GET /**: επιστρέφει το index.html. `STATIC_DIR = Path(__file__).parent / "static"` – διαδρομή σχετική με το ίδιο το αρχείο, άρα δουλεύει από οποιονδήποτε φάκελο εκκίνησης.
- Ο uvicorn ξεκινά με `uvicorn app.main:app`: «στο module app/main.py, πάρε τη μεταβλητή app».

Η διαδρομή ενός αιτήματος μέσα στην εφαρμογή

```
GET /videos/sample/seg_001.ts
  |
  ▼  uvicorn (θύρα 8000)
  ▼  metrics_middleware      → χρονόμετρο, in_progress +1
  ▼  router /videos          → get_video_file(video_id="sample", filename="seg_001.ts")
  ▼  video_service.resolve_video_file → έλεγχοι regex + path → (/app/media/hls/sample/seg_001.ts, video/
  mp2t)
  ▼  FileResponse + header X-Served-By: <όνομα Pod>
  ▼  tracked_body            → μετρά bytes ενώ στέλνει
  ▼  finally                 → bytes_total, duration, in_progress -1
```

6.9 backend/app/static/index.html – σελίδα demo

Ρόλος: μόνο η *δομή* της σελίδας (HTML). Χρησιμοποιείται για επίδειξη στον browser· η γεννήτρια φόρτου δεν τη χρειάζεται.

- `<meta charset>` και `viewport`: σωστή κωδικοποίηση και κλιμάκωση σε κινητά.
- `<select id="video-select">`: dropdown που γεμίζει το app.js από το GET /videos. `<button id="play-btn">`: ξεκινά την αναπαραγωγή.
- `<div id="status">`: μηνύματα κατάστασης. `<video controls muted>`: ο player· το `muted` επιτρέπει autoplay (οι browsers μπλοκάρουν autoplay με ήχο).
- `<div id="log">`: καταγραφή κάθε segment που κατέβηκε.
- Τα scripts μπαίνουν στο τέλος του body, ώστε τα στοιχεία να υπάρχουν όταν τρέξουν. Το **hls.js** (από CDN, έκδοση 1.5.17) φορτώνεται **πριν** το app.js, γιατί το app.js χρησιμοποιεί το global αντικείμενο `Hls`.

6.10 backend/app/static/app.js – λογική της σελίδας

- Στην αρχή παίρνει αναφορές στα στοιχεία της σελίδας με `getElementById`.
- **log(line)**: προσθέτει γραμμή στο log με `textContent` (όχι innerHTML – ασφαλές απέναντι σε XSS) και κάνει scroll στο τέλος.
- **loadVideos()** (async): `fetch("/videos")` → JSON → ένα `<option>` ανά βίντεο με value το URL της playlist. Γράφει πόσα βίντεο βρέθηκαν και ποιο instance απάντησε· απενεργοποιεί το κουμπί αν δεν υπάρχει βίντεο. Σφάλματα δικτύου πιάνονται με try/catch.
- **play(url)**: καθαρίζει το log και καταστρέφει τυχόν προηγούμενο player (`hls.destroy()`). Τρεις περιπτώσεις:
- 1) **Chrome/Firefox/Edge** (`Hls.isSupported()`): νέο `Hls()`, `loadSource` (κατεβάζει την playlist), `attachMedia(video)`. Το hls.js κατεβάζει μόνο του τα segments και τα δίνει στο video μέσω Media Source Extensions. Στο event `FRAG_LOADED` διαβάζει το header `X-Served-By` από το XHR, το μέγεθος (KB) και τον χρόνο λήψης (ms) και γράφει π.χ. «segment #3 410 KB 12 ms served by: video-streaming-abc». Στο event `ERROR` δείχνει μόνο τα fatal σφάλματα.
- 2) **Safari**: υποστηρίζει HLS εγγενώς, οπότε αρκεί `video.src = url` (χωρίς log ανά segment).
- 3) Καμία υποστήριξη: μήνυμα σφάλματος.
- `video.play().catch(() => {})`: ξεκινά την αναπαραγωγή και αγνοεί την απόρριψη αν ο browser δεν την επιτρέψει.
- Στο τέλος: σύνδεση του click στο κουμπί και κλήση `loadVideos()` μόλις ανοίξει η σελίδα.

Βλέποντας **διαφορετικά ονόματα Pods** στο log αποδεικνύεται στην πράξη ότι το Service μοιράζει τα αιτήματα στα αντίγραφα.

6.11 backend/app/static/style.css – εμφάνιση

Απλό σκούρο θέμα: κεντραρισμένη στήλη πλάτους 900px (`body`), βίντεο σε όλο το πλάτος με στρογγυλεμένες γωνίες, dropdown και κουμπί δίπλα-δίπλα με flexbox (`.controls`), γκρι γραμμή κατάστασης (`#status`), και log με monospace γραμματοσειρά, σταθερό ύψος 220px και κάθετη κύλιση (`#log`, `white-space: pre` για να κρατά τα κενά).

6.12 Dockerfile – κατασκευή του image

```
# Base image: official Python on a small Debian
FROM python:3.13-slim

# Python settings + where config.py finds the videos
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    MEDIA_DIR=/app/media

# Work inside /app
WORKDIR /app

# Install dependencies first (cached unless requirements.txt changes)
COPY backend/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Unprivileged user
RUN useradd --create-home --uid 1000 appuser

# Application code and prepared HLS videos
COPY backend/app ./app
COPY media/hls ./media/hls

# Run as the unprivileged user
USER appuser

# The server listens on 8000
EXPOSE 8000

# Start the server, reachable from outside the container
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Γραμμή	Εξήγηση
`FROM python:3.13-slim`	Επίσημη Python 3.13 πάνω σε ελαφρύ Debian. Το slim έχει μικρότερο μέγεθος και λιγότερα πακέτα.
`PYTHONDONTWRITEBYTECODE=1`	Η Python δεν γράφει αρχεία .pyc (άχρηστα σε container).
`PYTHONUNBUFFERED=1`	Τα logs βγαίνουν αμέσως (χωρίς buffer), ώστε να φαίνονται στο docker logs / kubectl logs.
`MEDIA_DIR=/app/media`	Λέει στο config.py πού είναι τα βίντεο μέσα στο image.
`WORKDIR /app`	Όλα γίνονται στον φάκελο /app.
`COPY requirements.txt + RUN pip install`	Πρώτα οι εξαρτήσεις ώστε να μένουν στην cache. `--no-cache-dir`: δεν κρατά την cache του pip (μικρότερο image).
`RUN useradd ... appuser`	Δημιουργεί χρήστη χωρίς δικαιώματα root, με UID 1000.
`COPY backend/app ./app`	Ο κώδικας → /app/app, γι' αυτό το `app.main:app` δουλεύει από το /app.
`COPY media/hls ./media/hls`	Τα έτοιμα HLS αρχεία «ψήνονται» μέσα στο image. Δεν χρειάζεται volume· κάθε Pod είναι αυτόνομο.
`USER appuser`	Το container τρέχει ως μη-root: αν παραβιαστεί η εφαρμογή, ο επιτιθέμενος έχει λιγότερα δικαιώματα.
`EXPOSE 8000`	Τεκμηρίωση ότι η εφαρμογή ακούει στο 8000.

Γραμμή	Εξήγηση
`CMD ["uvicorn", ...]`	Εκκίνηση server. `--host 0.0.0.0`: ακούει σε όλες τις διεπαφές, ώστε να είναι προσβάσιμος έξω από το container. Η μορφή JSON-array (exec form) τρέχει τον uvicorn ως PID 1 και λαμβάνει σωστά το SIGTERM του Kubernetes.

6.13 .dockerignore – τι ΔΕΝ μπαίνει στο build

```
.git
.gitignore
.venv
venv
**/__pycache__
**/*.pyc
**/.env
media/source
.vscode
.idea
*.pdf
Dockerfile
.dockerignore
```

Λειτουργεί όπως το .gitignore αλλά για το build context του Docker. Αποκλείει το ιστορικό git, virtual environments, cache Python, μυστικά `.env`, ρυθμίσεις IDE, PDF, και το ίδιο το Dockerfile. Ιδιαίτερα σημαντικό: `media/source` – τα αρχικά βίντεο (εδώ ~10 MB) δεν χρειάζονται στο image, μόνο τα HLS. Αποτέλεσμα: γρηγορότερο build, μικρότερο image, κανένα μυστικό μέσα του.

6.14 k8s/deployment.yaml – το Deployment

Ρόλος: λέει στον Kubernetes να τρέχει **3 πανομοιότυπα Pods** από το image `video-streaming:1.0`, μοιρασμένα στους κόμβους, με όρια πόρων και ελέγχους υγείας.

```
# deployment.yaml
# Runs the video streaming service as several identical Pods (replicas) spread over the nodes.
# Apply with:  kubectl apply -f k8s/

# API group and version for Deployments
apiVersion: apps/v1
# A Deployment keeps the requested number of Pods running and replaces them when they fail
kind: Deployment
metadata:
  # Name of the Deployment (used in kubectl commands)
  name: video-streaming
  # Labels: key/value tags used to find and group objects
  labels:
    app: video-streaming
spec:
  # How many Pods to run. The HPA will change this number automatically in step 5
  replicas: 3
  # Which Pods belong to this Deployment: the ones with the label app=video-streaming
  selector:
    matchLabels:
      app: video-streaming
  # The template every Pod is created from
  template:
    metadata:
      # Every Pod gets this label, so the selector above and the Service can find it
      labels:
        app: video-streaming
    spec:
      # Workload placement: spread the Pods as evenly as possible over the nodes
      topologySpreadConstraints:
        # At most 1 Pod difference between any two nodes
        - maxSkew: 1
```

```
      # "Spread over" = over different nodes (each node has a unique hostname label)
      topologyKey: kubernetes.io/hostname
      # If a perfect spread is impossible, still schedule the Pod (don't leave it Pending)
      whenUnsatisfiable: ScheduleAnyway
      # Count only our own Pods when computing the spread
      labelSelector:
        matchLabels:
          app: video-streaming
  containers:
    # Name of the container inside the Pod
    - name: video-streaming
      # The image built with: docker build -t video-streaming:1.0 .
      image: video-streaming:1.0
      # Use the image already loaded into the cluster; never try to download it from Docker Hub
      imagePullPolicy: IfNotPresent
      ports:
        # The port uvicorn listens on (see CMD in the Dockerfile). "http" is a name we can refer to
        - name: http
          containerPort: 8000
      # Resource management
      resources:
        # requests: what the scheduler reserves for the Pod; also the base for HPA percentages
        requests:
          cpu: 100m        # 0.1 CPU core
          memory: 128Mi
        # limits: the maximum the container may use (CPU is throttled, memory over the limit = restart)
        limits:
          cpu: 500m        # 0.5 CPU core
          memory: 256Mi
      # Readiness: send traffic to this Pod only when it can serve videos
      readinessProbe:
        httpGet:
          path: /health/ready
          port: http
        # Wait 2 s after start before the first check
        initialDelaySeconds: 2
        # Check every 5 s
        periodSeconds: 5
        # 3 failures in a row = not ready (removed from the Service)
        failureThreshold: 3
      # Liveness: restart the container if it stops answering
      livenessProbe:
        httpGet:
          path: /health/live
          port: http
        # Give the app time to start before the first check
        initialDelaySeconds: 10
        # Check every 10 s
        periodSeconds: 10
        # 3 failures in a row = restart
        failureThreshold: 3
```

Ανάλυση ανά ενότητα

- **apiVersion: apps/v1, kind: Deployment** – το είδος αντικειμένου.
- **metadata.name / labels** – το όνομα `video-streaming` (για `kubectl get deployment video-streaming`) και ετικέτα `app: video-streaming`.
- **spec.replicas: 3** – πόσα Pods. Αργότερα (βήμα 5) θα το αλλάζει αυτόματα ο HPA.
- **spec.selector.matchLabels** – «δικά μου είναι τα Pods με app=video-streaming». Πρέπει να ταιριάζει με το template.
- **spec.template** – το «καλούπι» από το οποίο φτιάχνεται κάθε Pod: έχει τα δικά του metadata (labels) και spec.
- **topologySpreadConstraints** (workload placement): `maxSkew: 1` = η διαφορά πλήθους Pods μεταξύ δύο κόμβων είναι το πολύ 1· `topologyKey: kubernetes.io/hostname` = μοίρασε ανά κόμβο (κάθε κόμβος έχει μοναδικό hostname label)· `whenUnsatisfiable:`

`ScheduleAnyway` = αν δεν γίνεται τέλεια κατανομή, τοποθέτησέ το ούτως ή άλλως (μην το αφήσεις Pending)· `labelSelector` = μέτρα μόνο τα δικά μας Pods. Έτσι με 3 κόμβους και 3 replicas βγαίνει ένα Pod ανά κόμβο, και αν πέσει ένας κόμβος χάνεται μόνο 1/3 της χωρητικότητας.

- **containers[0].image + imagePullPolicy: IfNotPresent** – χρησιμοποίησε το image που υπάρχει ήδη στον κόμβο (φορτωμένο με `minikube image load`), μην το ψάξεις στο Docker Hub (όπου δεν υπάρχει).
- **ports: name http, containerPort 8000** – η θύρα του uvicorn. Το όνομα `http` χρησιμοποιείται από probes και Service.
- **resources.requests** (100m CPU = 0,1 πυρήνα, 128Mi RAM): ο scheduler τοποθετεί το Pod μόνο σε κόμβο με τόσους ελεύθερους πόρους. Είναι και η βάση για τα ποσοστά του HPA (π.χ. «70% CPU» = 70m).
- **resources.limits** (500m CPU, 256Mi): ανώτατο όριο. Πάνω από το όριο CPU το container επιβραδύνεται (throttling)· πάνω από το όριο μνήμης σκοτώνεται (OOMKilled) και επανεκκινείται.
- **readinessProbe**: GET /health/ready, πρώτος έλεγχος 2 δευτ. μετά την εκκίνηση, κάθε 5 δευτ., 3 αποτυχίες → εκτός Service.
- **livenessProbe**: GET /health/live, ξεκινά στα 10 δευτ. (χρόνος για εκκίνηση), κάθε 10 δευτ., 3 αποτυχίες → επανεκκίνηση container.

Μονάδα	Σημασία
`1` / `1000m`	1 πλήρης πυρήνας CPU (m = millicores, χιλιοστά του πυρήνα)
`100m`	0,1 πυρήνα
`128Mi`	128 mebibytes = 128 × 1024 × 1024 bytes (Mi, Gi = δυνάμεις του 2· M, G = δυνάμεις του 10)

6.15 k8s/service.yaml – το Service

Ρόλος: δίνει σε όλα τα Pods **μία σταθερή διεύθυνση** και μοιράζει τα αιτήματα (load balancing) μεταξύ όσων είναι READY.

```
# service.yaml
# Gives all video-streaming Pods ONE stable address and spreads the requests over them.

# API version for core objects such as Services
apiVersion: v1
# A Service is a stable network endpoint in front of a changing set of Pods
kind: Service
metadata:
  # Name of the Service. Inside the cluster it is also its DNS name: http://video-streaming
  name: video-streaming
  labels:
    app: video-streaming
spec:
  # NodePort: reachable from outside the cluster on a fixed port of every node
  type: NodePort
  # Send traffic to every READY Pod with this label
  selector:
    app: video-streaming
  ports:
    - name: http
      # Port of the Service inside the cluster
      port: 80
      # Port on the Pod (the named port "http" = 8000 in the Deployment)
      targetPort: http
      # Fixed port on every node (allowed range 30000-32767)
```

```
    nodePort: 30080
```

- **apiVersion: v1, kind: Service** – αντικείμενο της core ομάδας.
- **metadata.name** – είναι και το DNS όνομα μέσα στο cluster: άλλα Pods μπορούν να καλούν `http://video-streaming`.
- **type: NodePort** – εκτός από εσωτερική IP, ανοίγει τη θύρα 30080 σε κάθε κόμβο, άρα η υπηρεσία είναι προσβάσιμη απ' έξω.
- **selector: app: video-streaming** – ποια Pods βρίσκονται πίσω από το Service. Ο Kubernetes διατηρεί αυτόματα τη λίστα **Endpoints** με τις IP μόνο των READY Pods.
- **Οι τρεις θύρες:**

```
client ──► <IP κόμβου>:30080  ──►  Service :80  ──►  Pod :8000 (uvicorn)
           nodePort                 port              targetPort: http (= containerPort 8000)
```

Το `targetPort: http` αναφέρεται στο *όνομα* της θύρας του Deployment. Αν αλλάξει ο αριθμός θύρας στο Deployment, το Service συνεχίζει να δουλεύει χωρίς αλλαγή. Η κατανομή γίνεται από το **kube-proxy** σε κάθε κόμβο (κανόνες iptables/IPVS), ουσιαστικά τυχαία ανά σύνδεση.

Επειδή το HLS κάνει πολλά μικρά, ανεξάρτητα αιτήματα (ένα ανά segment) και η εφαρμογή είναι stateless, διαδοχικά segments του ίδιου βίντεο μπορούν να έρθουν από διαφορετικά Pods χωρίς κανένα πρόβλημα.

6.16 scripts/locustfile.py – γεννήτρια φόρτου

Ρόλος: σενάριο για το εργαλείο **Locust** που προσομοιώνει πολλούς ταυτόχρονους θεατές (βήμα 4), ώστε να μετρηθεί η απόδοση και να ενεργοποιηθεί ο autoscaler.

```
import time
from locust import HttpUser, task, between

class VideoStreamingUser(HttpUser):
    # Περιμένει 1-2 δευτερόλεπτα πριν ξεκινήσει (σαν να περιηγείται στη σελίδα)
    wait_time = between(1, 2)

    @task
    def watch_video(self):
        # 1. Ζητάει τη λίστα με τα διαθέσιμα βίντεο
        response = self.client.get("/videos", name="/videos")
        if response.status_code != 200:
            return

        data = response.json()
        videos = data.get("videos", [])
        if not videos:
            return

        # Επιλέγουμε το πρώτο διαθέσιμο βίντεο από τη λίστα
        video = videos[0]
        video_id = video["id"]
        playlist_url = video["playlist"]

        # 2. Κατεβάζει το αρχείο playlist (.m3u8)
        playlist_response = self.client.get(playlist_url, name="/videos/{id}/index.m3u8")
        if playlist_response.status_code != 200:
            return

        # 3. Αναλύει το .m3u8 για να βρει τα ονόματα των αρχείων .ts
        segments = []
        for line in playlist_response.text.splitlines():
            line = line.strip()
            # Απομονώνει τις γραμμές που δεν είναι σχόλια/metadata (δεν ξεκινάνε με #)
            if line and not line.startswith('#'):
                segments.append(line)
```

```
        # 4. Προσομοίωση αναπαραγωγής HLS: Κατεβάζει τα segments ένα-ένα
        for segment in segments:
            segment_url = f"/videos/{video_id}/{segment}"
            self.client.get(segment_url, name="/videos/{id}/seg_NNN.ts")

            # Το αρχείο prepare_media.py φτιάχνει segments των 4 δευτερολέπτων από προεπιλογή
            # Ο client "περιμένει" 4 δευτερόλεπτα (αναπαραγωγή) πριν ζητήσει το επόμενο κομμάτι.
            time.sleep(4)
```

- **class VideoStreamingUser(HttpUser)**: κάθε εικονικός χρήστης είναι ένα αντικείμενο αυτής της κλάσης. Το `self.client` είναι HTTP client (βασισμένος στο requests) που καταγράφει αυτόματα χρόνους και σφάλματα.
- `wait_time = between(1, 2)`: παύση 1–2 δευτ. ανάμεσα στις επαναλήψεις του task.
- **@task watch_video**: (1) GET /videos, (2) παίρνει το πρώτο βίντεο και κατεβάζει την playlist, (3) διαβάζει τις γραμμές που δεν ξεκινούν με # = ονόματα segments, (4) κατεβάζει κάθε segment και περιμένει 4 δευτ., όσο κρατά η αναπαραγωγή του, όπως ένας πραγματικός player.
- Η παράμετρος `name=` ομαδοποιεί όλα τα segments κάτω από ένα όνομα στα στατιστικά του Locust, αντί για μία γραμμή ανά αρχείο.
- Εκτέλεση: `locust -f scripts/locustfile.py --host http://<URL-του-service>` και άνοιγμα του http://localhost:8089 για να ορίσεις πλήθος χρηστών.

> Δύο παρατηρήσεις: (α) Το `locust` δεν είναι στο requirements.txt (σωστά, αφού δεν χρειάζεται στον server)· εγκαθίσταται χωριστά με `pip install locust`. (β) Το `time.sleep(4)` είναι σταθερό ενώ το βίντεο sample έχει segments ~8,3 δευτ.· για ρεαλιστικότερη προσομοίωση θα μπορούσε να διαβάζεται η διάρκεια από τις γραμμές `#EXTINF`. Το `time.sleep` δεν μπλοκάρει τους άλλους χρήστες, επειδή το Locust κάνει monkey-patch μέσω gevent.

6.17 media/ – τα βίντεο

Διαδρομή	Περιεχόμενο
`media/source/earth_spinning.mp4`	Αρχικό βίντεο (~9,8 MB). Είσοδος για το prepare_media.py. Δεν μπαίνει στο image (.dockerignore).
`media/hls/sample/`	index.m3u8 + 4 segments (seg_000–003.ts, ~1,7 MB συνολικά, ~30 δευτ.).
`media/hls/test/`	index.m3u8 + 15 segments των 4 δευτ. (το 60-δευτερόλεπτο test pattern του --test).

Κάθε υποφάκελος του `media/hls/` που περιέχει `index.m3u8` εμφανίζεται αυτόματα ως βίντεο στο GET /videos – το όνομα του φακέλου είναι το id.

> Το README αναφέρει ότι ο φάκελος media/ «δεν είναι στο git», αλλά στην τρέχουσα κατάσταση του repository τα αρχεία του είναι committed και το .gitignore δεν τον αποκλείει.

6.18 .gitignore, README.md, LICENSE

- **.gitignore**: λέει στο git να αγνοεί cache Python (`__pycache__/`, `*.pyc`), virtual environments, το `.env` με μυστικά και αρχεία IDE/λειτουργικού (`.vscode`, `.idea`, `.DS_Store`, `Thumbs.db`).
- **README.md**: η τεκμηρίωση του έργου – κατάσταση βημάτων (1–2 ολοκληρωμένα, 3 σε εξέλιξη, 4–6 προς υλοποίηση), αρχιτεκτονική, οδηγίες για προετοιμασία βίντεο, Docker,

minikube, ρυθμίσεις, μετρικές και αντιμετώπιση προβλημάτων.

- **LICENSE**: GNU GPL v3 – ελεύθερη χρήση, τροποποίηση και διανομή, αρκεί τα παράγωγα έργα να διανέμονται επίσης υπό GPL με τον πηγαίο κώδικα.

7. Όλη η ροή από την αρχή μέχρι το τέλος

```
1. python scripts/prepare_media.py --test        → media/hls/test/index.m3u8 + seg_*.ts
2. docker build -t video-streaming:1.0 .          → image με Python, κώδικα και HLS αρχεία
3. minikube start --nodes 3 --driver=docker      → cluster 3 κόμβων
4. minikube image load video-streaming:1.0        → το image αντιγράφεται σε κάθε κόμβο
5. kubectl apply -f k8s/                          → Deployment (3 Pods) + Service (NodePort 30080)
      ├─ scheduler: τοποθετεί 1 Pod ανά κόμβο (topologySpreadConstraints)
      ├─ kubelet: ξεκινά container → uvicorn :8000
      ├─ readinessProbe /health/ready = 200 → το Pod μπαίνει στα Endpoints του Service
      └─ livenessProbe  /health/live  = 200 → δεν γίνεται επανεκκίνηση
6. minikube service video-streaming --url        → URL προς το Service
7. Browser: GET /  → index.html → app.js → GET /videos → hls.js → playlist → segments
      κάθε segment μπορεί να έρθει από άλλο Pod (header X-Served-By)
8. Locust: εκατοντάδες εικονικοί θεατές → μετρικές στο /metrics → (μελλοντικά) HPA + Prometheus
```

7.1 Χρήσιμες εντολές

Σκοπός	Εντολή
Pods και κόμβοι	`kubectl get pods -o wide`
Logs ενός Pod	`kubectl logs <pod>`
Γιατί δεν ξεκινά ένα Pod	`kubectl describe pod <pod>`
Χειροκίνητη κλιμάκωση	`kubectl scale deployment video-streaming --replicas=5`
Επανεκκίνηση όλων των Pods	`kubectl rollout restart deployment video-streaming`
Ενημέρωση σε νέα έκδοση	νέο tag (1.1) → `minikube image load` → αλλαγή image στο YAML → `kubectl apply -f k8s/`
Αφαίρεση εφαρμογής	`kubectl delete -f k8s/`
Διακοπή / διαγραφή cluster	`minikube stop` / `minikube delete`

7.2 Συχνά προβλήματα

Σύμπτωμα	Αιτία και λύση
`ErrImagePull` / `ImagePullBackOff`	Ο κόμβος δεν έχει το image → `minikube image load video-streaming:1.0`, έλεγχος tag στο YAML.
`CrashLoopBackOff`	Το container ξεκινά και πεθαίνει → `kubectl logs <pod>`.
`READY 0/1`	Αποτυγχάνει το readiness: το image δεν έχει βίντεο. Τρέξε prepare_media.py, ξαναχτίσε, ξαναφόρτωσε, rollout restart.
`Pending`	Δεν υπάρχουν αρκετοί πόροι για τα requests → `kubectl describe pod`, δώσε περισσότερη μνήμη στο minikube/Docker.
YAML error: «mapping values are not allowed»	Λάθος εσοχή ή λείπει κενό μετά το «:». Έλεγξε για Tabs.
«selector does not match template labels»	Το spec.selector.matchLabels διαφέρει από τα template.metadata.labels.